

```
"""
```

AutoGen Multi-Agent System Example

This example demonstrates multiple agents collaborating to research, plan, and write content.

```
"""
```

```
import autogen
from autogen import AssistantAgent, UserProxyAgent, GroupChat, GroupChatManager
```

```
# Configuration for the LLM
```

```
config_list = [
    {
        "model": "gpt-4",
        "api_key": "your-api-key-here", # Replace with your actual API key
    }
]
```

```
llm_config = {
    "config_list": config_list,
    "temperature": 0.7,
    "timeout": 120,
}
```

```
# 1. User Proxy Agent - Represents the human user and executes code
```

```
user_proxy = UserProxyAgent(
    name="User_Proxy",
    system_message="A human admin who can execute code and provide feedback.",
    code_execution_config={
```

```
        "work_dir": "coding",
        "use_docker": False, # Set to True if you have Docker installed
    },
    human_input_mode="TERMINATE", # Can be "ALWAYS", "TERMINATE", or "NEVER"
)
```

2. Researcher Agent - Gathers information and does research

```
researcher = AssistantAgent(
    name="Researcher",
    system_message="""You are a research specialist. Your role is to:
    - Gather relevant information on the given topic
    - Identify key facts and data points
    - Provide comprehensive research summaries
    - Cite sources when possible
    Always start your response with 'RESEARCH FINDINGS:'''",
    llm_config=llm_config,
)
```

3. Planner Agent - Creates structured plans

```
planner = AssistantAgent(
    name="Planner",
    system_message="""You are a strategic planner. Your role is to:
    - Analyze research findings
    - Create structured outlines and plans
    - Break down complex tasks into manageable steps
    - Ensure logical flow and organization
    Always start your response with 'PLAN:'''",
    llm_config=llm_config,
```

)

4. Writer Agent - Creates content based on research and plans

```
writer = AssistantAgent(  
    name="Writer",  
    system_message="""You are a professional content writer. Your role is to:  
    - Transform research and plans into well-written content  
    - Maintain clear, engaging, and informative style  
    - Ensure accuracy and coherence  
    - Polish and refine the final output  
    Always start your response with 'CONTENT:'''",  
    llm_config=llm_config,  
)
```

5. Critic Agent - Reviews and provides feedback

```
critic = AssistantAgent(  
    name="Critic",  
    system_message="""You are a critical reviewer. Your role is to:  
    - Review content for accuracy, clarity, and completeness  
    - Identify gaps or areas for improvement  
    - Provide constructive feedback  
    - Approve final output when satisfactory  
    Always start your response with 'REVIEW:' and end with either 'APPROVED' or  
    'NEEDS REVISION'''",  
    llm_config=llm_config,  
)
```

6. Coder Agent - Writes and debugs code

```
coder = AssistantAgent(  
    name="Coder",  
    system_message="""You are an expert programmer. Your role is to:  
    - Write clean, efficient, and well-documented code  
    - Debug and fix issues  
    - Follow best practices  
    - Provide explanations for your code  
    Always start your response with 'CODE: '""",  
    llm_config=llm_config,  
)
```

```
def create_group_chat_example():  
    """Example 1: Group Chat with multiple agents collaborating"""  
  
    # Create a group chat with all agents  
    groupchat = GroupChat(  
        agents=[user_proxy, researcher, planner, writer, critic],  
        messages=[],  
        max_round=12,  
        speaker_selection_method="auto", # Can be "auto", "manual", or  
        "round_robin"  
    )  
  
    # Create a manager to coordinate the group chat  
    manager = GroupChatManager(groupchat=groupchat, llm_config=llm_config)  
  
    # Start the conversation
```

```
user_proxy.initiate_chat(
    manager,
    message="""Create a comprehensive blog post about the benefits of
    artificial intelligence in healthcare. The post should be well-researched,
    structured, and ready for publication."""
)
```

```
def create_sequential_chat_example():
    """Example 2: Sequential chats - agents talking in sequence"""

    task = "Analyze the impact of remote work on productivity and create a summary
    report."

    # Step 1: User asks Researcher
    print("\n=== STEP 1: Research Phase ===")
    user_proxy.initiate_chat(
        researcher,
        message=f"Please research the following topic: {task}",
        max_turns=2,
    )

    # Step 2: Planner creates structure based on research
    print("\n=== STEP 2: Planning Phase ===")
    user_proxy.initiate_chat(
        planner,
        message="Based on the research above, create a structured outline for the
    report.",
    )
```

```

        max_turns=2,
    )

    # Step 3: Writer creates content
    print("\n=== STEP 3: Writing Phase ===")
    user_proxy.initiate_chat(
        writer,
        message="Using the research and plan above, write the complete report.",
        max_turns=2,
    )

    # Step 4: Critic reviews
    print("\n=== STEP 4: Review Phase ===")
    user_proxy.initiate_chat(
        critic,
        message="Please review the report and provide feedback.",
        max_turns=2,
    )

def create_coding_task_example():
    """Example 3: Coding task with code execution"""

    # Create a simple group for coding task
    coding_groupchat = GroupChat(
        agents=[user_proxy, coder],
        messages=[],
        max_round=6,

```

```
)
```

```
manager = GroupChatManager(groupchat=coding_groupchat, llm_config=llm_config)
```

```
user_proxy.initiate_chat(
```

```
    manager,
```

```
    message="""Write a Python function that:
```

```
    1. Takes a list of numbers as input
```

```
    2. Calculates the mean, median, and mode
```

```
    3. Returns the results as a dictionary
```

```
    4. Includes proper error handling
```

```
    Then execute the code to demonstrate it works."""
```

```
)
```

```
def create_two_agent_chat():
```

```
    """Example 4: Simple two-agent conversation"""
```

```
    # Researcher talks to Writer directly
```

```
    researcher.initiate_chat(
```

```
        writer,
```

```
        message="""I've found that AI in healthcare can:
```

```
        1. Improve diagnostic accuracy by 30%
```

```
        2. Reduce administrative costs by 40%
```

```
        3. Enable personalized treatment plans
```

```
        Please write a compelling introduction paragraph about these benefits."""
```

```
        max_turns=2,  
)
```

```
# Main execution
```

```
if __name__ == "__main__":
```

```
    print("=" * 80)
```

```
    print("AutoGen Multi-Agent System Examples")
```

```
    print("=" * 80)
```

```
# Choose which example to run
```

```
print("\nSelect an example to run:")
```

```
print("1. Group Chat - Multiple agents collaborating on a blog post")
```

```
print("2. Sequential Chat - Agents working in sequence on a report")
```

```
print("3. Coding Task - Coder agent writing and executing code")
```

```
print("4. Two-Agent Chat - Simple conversation between two agents")
```

```
choice = input("\nEnter your choice (1-4): ")
```

```
if choice == "1":
```

```
    create_group_chat_example()
```

```
elif choice == "2":
```

```
    create_sequential_chat_example()
```

```
elif choice == "3":
```

```
    create_coding_task_example()
```

```
elif choice == "4":
```

```
    create_two_agent_chat()
```

```
else:
```



```
print("Invalid choice. Running default example (Group Chat)...")
create_group_chat_example()

print("\n" + "=" * 80)
print("Execution completed!")
print("=" * 80)
```